



10_1

Data Scientist

Numpy

NumPy (Numerical Python) es una biblioteca fundamental para la computación científica en Python. Proporciona soporte para arreglos y matrices multidimensionales, junto con una colección de funciones matemáticas para operar con estos arreglos de manera eficiente. NumPy es ampliamente utilizado en diversas áreas, incluidas las matemáticas, la física, la ingeniería, el aprendizaje automático y la ciencia de datos.

www.opentowork.es



#04/24 mihifidem



Numpy

NumPy (**N**umerical **P**ython) es una librería numérica de Python, de código abierto, que proporciona estructuras de datos matriciales y funciones matemáticas de alto nivel. Si te quieres dedicar a la ciencia de datos, estás siguiendo una ingeniería o algo relacionado con matemáticas, NumPy será la base de todo lo que hagas.

Cómo instalar NumPy

Si vas a trabajar con Notebooks en la nube, NumPy ya estará instalado. Pero si vas a trabajar en local, puedes instalarla desde la consola en tu máquina directamente o en un ambiente virtual de la siguiente manera:

```
pip install numpy
```

Por convención, se importa a Python de esta manera:

```
import numpy as np
```

¿Por qué usar NumPy?

Tiene una estructura de datos especial que te permite representar elementos complejos como imágenes, videos, canciones, modelos 3D, etc. Si tienes datos por procesar, NumPy será tu mejor amigo. Además, podrás realizar operaciones numéricas de manera muy sencilla, ahorrándote tener que programar ciclos o cosas extrañas.

A simple vista podrá parecer que con las listas de Python es posible hacer las mismas cosas que con NumPy, pero hay 3 grandes diferencias:

1. Las estructuras de datos de NumPy ocupan menos espacio en memoria que las listas.
2. También son significativamente más rápidas. Esto es porque NumPy está escrito en su mayor parte en C. Así que es tu aliado para lidiar con **grandes cantidades de datos**.
3. En NumPy ya están realizadas muchas funciones que en Python tendríamos que programar con ciclos y dominando muchos conceptos matemáticos.

Pero eso no es todo, varias de las grandes librerías de Python para Data Science, como Pandas, están escritas sobre NumPy y lo potencian. De hecho NumPy es la base de [SciPy](#).

Numpy

Arrays de NumPy

En NumPy se trabaja con una estructura de datos llamada array o arreglos numéricos multidimensionales. Los arrays son parecidos a las **listas de Python**, por ejemplo, heredan algunas propiedades como el ser mutables y poder realizar slicing (de hecho podemos crear un array a partir de una lista). Pero tienen diferencias importantes: son menos pesados, más rápidos y permiten crear fácilmente arrays n-dimensionales.

¿N-dimensionales? ¿Qué significa eso? Bueno, vamos a darles nombres y una grafiquita para que quede más claro. Un array **unidimensional** puede ser una fila o una columna de una tabla (sí, igual que una lista), esta se conoce como **vector**. Un array **bidimensional** es lo que llamamos comúnmente **matriz**. Y un array de 3 dimensiones (o más), es decir, una matriz de matrices, se conoce como **tensor**.



Numpy

Pero... ¿Cómo se ve esto en código? *(Ignora cómo lo construí, más adelante te enseñaré a hacerlo)*

```
>>> vector = np.arange(5)
>>> vector
array([0, 1, 2, 3, 4])

>>> matrix = np.arange(9).reshape(3, 3)
>>> matrix
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])

>>> tensor = np.arange(12).reshape(3, 2, 2)
>>> tensor
array([[[ 0,  1],
        [ 2,  3]],

       [[ 4,  5],
        [ 6,  7]],

       [[ 8,  9],
        [10, 11]]])
```

🔗 Algo muy importante que tienes que saber es que los arrays tienen que tener un **único tipo de dato**. Esto hace que sea muy eficiente almacenarlos y acceder a ellos. Si intentas poner varios tipos de datos, NumPy te cambiará algunos elementos para que todos sean homogéneos. Por ejemplo, si creas un array con strings y enteros, te los transformará para que todos sean strings o si pones enteros y flotantes, hará que todos sean flotantes.

Y una última regla: el tamaño de tus arrays 2D+ tiene que ser “rectangular”. Esto es igual que en matemáticas, si tu primera columna tiene 4 elementos, todas las columnas deberán tener 4 elementos. Lo mismo aplica para las filas.

Numpy

¿Cómo crear un array en NumPy?

Como ya lo mencioné, se puede crear un array a partir de una lista o una lista de listas. Así que la forma más directa de crearlo es con `np.array(tu_lista)`. Y al igual que las listas, puedes acceder a sus valores por medio de sus **índices**.

```
>>> my_first_vector = np.array([2, 5, 6, 23])
>>> print(my_first_vector)
[ 2  5  6 23]

>>> my_first_matrix = np.array([[2, 4], [6, 8]])
>>> print(my_first_matrix)
[[2 4]
 [6 8]]

>>> my_list = [0, 1, 2, 3, 4]
>>> print(np.array(my_list))
[0 1 2 3 4]
```

Creando arrays secuenciales con `arange` y `linspace`

También puedes crear arrays con secuencias de números. Te mostraré 2 maneras de hacerlo.

`np.arange()`

Funciona parecido a `range` en Python, solo que en lugar de regresar un generador, retorna un *ndarray*. Sus argumentos principales son: `start`, `stop` y `step`. Con lo que puedes dar un rango de valores y cada cuánto quieres que aparezcan.

Toma en cuenta que `start` es inclusivo y `stop` es exclusivo. Igual que al hacer slicing en una lista.

```
>>> print(np.arange(start=2, stop=10, step=2))
[2 4 6 8]

>>> print(np.arange(11, 1, -2))
[11 9 7 5 3]
```

Numpy

`np.linspace()`

Con `np.arange()` puedes decir cada cuánto quieres que genere los elementos y con ello te dará un tamaño de array. En cambio con `np.linspace()` puedes decir el **tamaño del array** (num) y los *steps* se calcularán automáticamente. Y aquí tanto start como stop son inclusivos.

```
>>> print(np.linspace(start=11, stop=12, num=18))  
[11.      11.05882353 11.11764706 11.17647059 11.23529412 11.29411765  
 11.35294118 11.41176471 11.47058824 11.52941176 11.58823529 11.64705882  
 11.70588235 11.76470588 11.82352941 11.88235294 11.94117647 12.      ]  
  
>>> print(np.linspace(0, 1, 11))  
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
```

Creando arrays vacíos y predefinidos

Es posible que necesites crear arreglos “vacíos” o con valores pre-definidos. Para ello hay 3 funciones muy útiles:

`np.zeros()` y `np.ones()`

Ambas funciones te permiten crear arrays llenos de **0** o **1** según corresponda. Toma en cuenta que si quieres crear más de una dimensión, deberás pasarlo en una tupla.

```
>>> print(np.zeros(4))  
[0. 0. 0. 0.]  
  
>>> print(np.zeros((2, 2)))  
[[0. 0.]  
 [0. 0.]]  
  
>>> print(np.ones(6))  
[1. 1. 1. 1. 1. 1.]
```

Numpy

np.full()

Crea un array con **un valor en específico**. Tiene 2 argumentos principales: shape que tiene que ser pasado como una tupla con las dimensiones y fill_value con el valor que desees.

```
>>> print(np.full(shape=(2, 2), fill_value=5))
```

```
[[5 5]
 [5 5]]
```

```
>>> print(np.full((2, 3, 4), 0.55))
```

```
[[[0.55 0.55 0.55 0.55]
  [0.55 0.55 0.55 0.55]
  [0.55 0.55 0.55 0.55]]
```

```
[[0.55 0.55 0.55 0.55]
 [0.55 0.55 0.55 0.55]
 [0.55 0.55 0.55 0.55]]]
```

np.full_like()

Sirve si ya tienes un array creado y quieres tomarlo como base para crear otro con el mismo tamaño, pero con un mismo valor.

```
>>> base = np.linspace(2, 6, 4)
```

```
>>> print(np.full_like(base, np.pi))
```

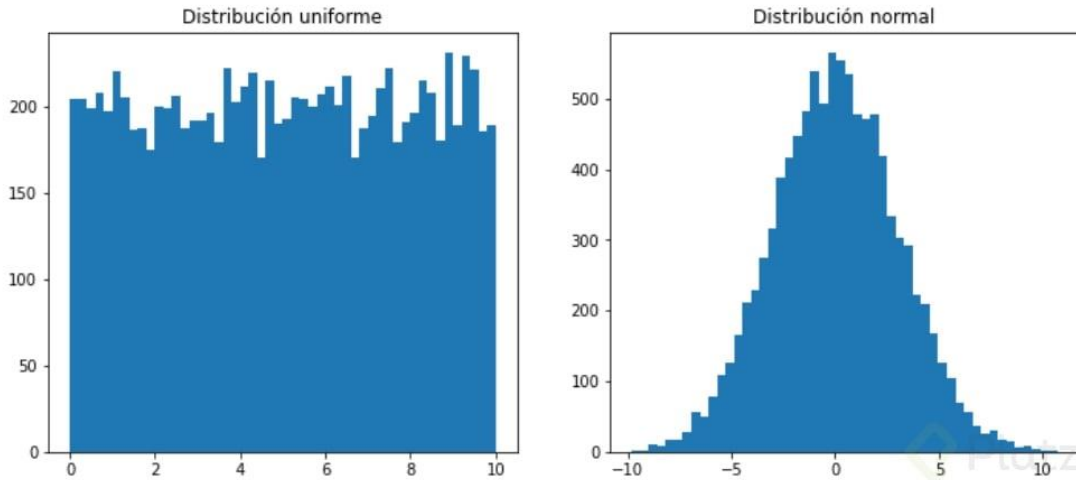
```
[3.14159265 3.14159265 3.14159265 3.14159265]
```

Arreglos aleatorios

Si quieres generar una serie de números totalmente aleatoria, ¡lo puedes hacer! Pero no solo eso, sino que si quieres que esos números obedezcan una distribución, puedes indicarlo. Para ello deberás usar np.random. Veamos algunas funciones que te pueden ayudar.

- .rand(): números aleatorios en una **distribución uniforme**. Permite crear ndarrays.
- .uniform(): parecido a rand, pero permite ingresar los límites de la muestra.
- .randn(): números aleatorios en una **distribución normal**. Permite crear ndarrays.
- .normal(): parecido a randn, pero permite escalar los límites de la muestra.
- .randint(): números enteros aleatorios entre un rango dado.

Numpy



```
>>> print(np.random.rand(2, 2))
[[0.39449172 0.53571876]
 [0.02221565 0.74657571]]

>>> print(np.random.uniform(low=0, high=1, size=6))
[0.13078989 0.34315991 0.77526217 0.24214904 0.69214014 0.25820501]

>>> print(np.random.randn(2, 2))
[[-0.23459831 -1.62879852]
 [-1.77888446 -1.09441255]]

>>> print(np.random.normal(loc=0, scale=2, size=6))
[ 1.62808825 -2.00185168  1.7059492  0.78900208 -0.92712749  0.1071917 ]

>>> print(np.random.randint(low=0, high=10, size=(3, 3)))
[[2 0 3]
 [7 7 1]
 [4 1 2]]
```

Numpy

Tamaño de los arrays

Hay un par de funciones y métodos que te ayudarán a saber las dimensiones que un array y su tipo de datos:

- `.shape`: devuelve las dimensiones del array. Para una matriz el primer valor son las filas y el segundo las columnas. Para un tensor el primer valor es la “profundidad”, el segundo las filas y el tercero las columnas.
- `.reshape()`: redimensiona un array, es decir, te permite crear matrices o tensores a partir de vectores y viceversa. También funciona como método.
- `.dtype`: devuelve el tipo de dato de un array.

```
>>> C = np.arange(1, 9).reshape(2, 2, 2)
```

```
print(C)
```

```
[[[1 2]
```

```
 [3 4]]
```

```
 [[5 6]
```

```
 [7 8]]]
```

```
>>> a = np.arange(1,10)
```

```
>>> B = np.reshape(a, [3,3])
```

```
>>> print(B)
```

```
[[1 2 3]
```

```
 [4 5 6]
```

```
 [7 8 9]]
```

```
>>> print(B.shape)
```

```
(3, 3)
```

```
>>> print(B.dtype)
```

```
int64
```

Numpy

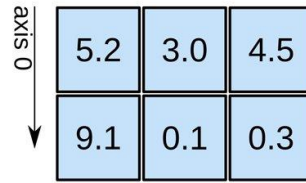
1D array



axis 0

shape: (4,)

2D array

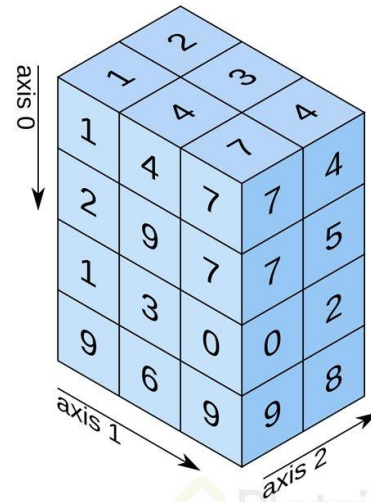


axis 0

axis 1

shape: (2, 3)

3D array



axis 0

axis 1

axis 2

shape: (4, 3, 2)

Numpy

Manipulando arrays

Es hora de manipular los arrays. De manera muy parecida a las listas, los ndarrays permiten hacer slicing con la notación [filas, columnas] y sus índices inician en 0.

Se puede hacer slicing con esta notación [2:5, 3:6]. Para seleccionar toda una columna o fila, se usa :.

Por ejemplo:

- [1, :] ← esto selecciona todas las columnas de la segunda fila.
- [:, 2] ← esto selecciona todas las filas de la tercera columna.

Veamos algunos ejemplos con una matriz para que quede más claro.

```
>>> matrix_cool = np.arange(9).reshape(3, 3)
>>> print(matrix_cool)
[[0 1 2]
 [3 4 5]
 [6 7 8]]

>>> print(matrix_cool[1, 2])
5

>>> print(matrix_cool[0, :])
[0 1 2]

>>> print(matrix_cool[:, 1])
[1 4 7]

>>> print(matrix_cool[:, 1:])
[[1 2]
 [4 5]
 [7 8]]

>>> print(matrix_cool[0:2, 0:2])
[[0 1]
 [3 4]]

>>> print(matrix_cool[:, :])
[[0 1 2]
 [3 4 5]
 [6 7 8]]
```

Numpy

Copiar un array de NumPy

Al ser mutables, para copiar un ndarray lo mejor es usar `array1.copy()` o nuevamente `np.array(array1)`.

 Incorrecto

```
>>> a1 = np.array([2, 4, 6])
>>> a2 = a1
>>> a1[0] = 8
>>> print(a1)
[8 4 6]
>>> print(a2)
[8 4 6]
```

 Correcto

```
>>> a1 = np.array([2, 4, 6])
>>> a2 = a1.copy()
>>> a1[0] = 8
>>> print(a1)
[8 4 6]
>>> print(a2)
[2 4 6]
```

Numpy

Funciones matemáticas

¡Has aprendido un montón hasta ahora! Así que vamos a terminar revisando un poco sobre las posibilidades que nos da NumPy con las operaciones matemáticas.

Empecemos con la **suma** y **resta**. Si intentas sumar dos listas, se van a concatenar. Pero si intentas sumar 2 ndarrays del mismo tamaño, se sumaran cada uno de sus elementos entre sí sin problemas, al igual que hacer una suma de matrices.

```
>>> A = np.arange(5, 11)
>>> print(A)
[ 5  6  7  8  9 10]
>>> print(A + 10)
[15 16 17 18 19 20]

>>> B = np.full(4, 3)
>>> C = np.ones(4, dtype='int')
>>> print(B)
[3 3 3 3]
>>> print(C)
[1 1 1 1]
>>> print(B - C)
[2 2 2 2]
```

Con la **multiplicación** y **división** las cosas se ponen un poco más complejas. Si tienes una matriz y la multiplicas o divides por un escalar, las operaciones se aplicarán de manera muy similar que lo que vimos con la suma.

Pero si intentas multiplicar una matriz y un vector o dos matrices, las reglas cambian, y se aplica algo llamado *broadcasting* y aparecen operaciones como el *producto punto*. Con NumPy es extremadamente sencillo aplicarlo, pero debes tener claro los conceptos matemáticos detrás de ello. No te preocupes, más adelante hablaré de eso, ahora veamos operaciones básicas.

```
>>> print(A * 10)
[ 50  60  70  80  90 100]

>>> print(A / 10)
[0.5 0.6 0.7 0.8 0.9 1. ]
```

Numpy

Algo muy útil que seguramente usarás en el día a día es la **estadística**. Para obtener valores como la media, mediana o desviación estándar. Solo necesitas un array con tus datos y NumPy hará el resto por ti.

```
>>> height_list = [74, 74, 72, 72, 73, 69, 69, 71, 76, 71, 73, 73, 74, 74, 69, 70, 73, 75, 78, 79, 76, 74, 76, 72, 71, 75]
>>> print(np.mean(height_list)) # media
73.1923076923077

>>> print(np.median(height_list)) # mediana
73.0

>>> print(np.std(height_list)) # desviación estándar
2.572326554954764

>>> print(np.max(height_list)) # máximo
79

>>> print(np.min(height_list)) # mínimo
69
```

Como puedes ver, hacer operaciones matemáticas es extremadamente sencillo y muy diferente a hacerlo con listas de Python. Además, en NumPy encontrarás muchas otras funcionalidades como trigonometría, logaritmos, valores absolutos, exponenciales, constantes universales, etc.

Numpy

Acceder a los elementos de un array.

`[], [], ...`

Acceder a un elemento de un array.

Para acceder a un elemento se utiliza la referencia a su posición dentro del array mediante uno o más **índices** según se trate de un array unidimensional o de más dimensiones, escribiendo el índice o índices entre corchetes "`[]`" y separando con comas "`,`" cuando sean más de uno: `[1]`, `[5, 23]`, `[-1, 1, 10]`.

En cada **eje** o dimensión el índice 0 hace referencia al primer elemento, el 1 al segundo y así sucesivamente hasta el último. También es posible referenciar comenzando desde el último elemento utilizando números negativos: en ese caso el índice -1 hace referencia al último elemento, el -2 al penúltimo y así sucesivamente.

Cuando se hace referencia a un índice inexistente en un array Numpy genera la excepción **IndexError**.

```
# Acceder a los elementos de un array.

# Declarar 3 arrays con distinto número de dimensiones:

a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(a)

# [1 2 3 4 5]

print(b)

# [[1 2 3]
#  [4 5 6]]

print(c)

# [[[1 2]
#    [3 4]]
#
#  [[5 6]
#    [7 8]]]

# Acceder al primer elemento de cada array:

print(a[0])

# 1
```

Numpy

```
print(b[0,0])  
  
# 1  
  
print(c[0,0,0])  
  
# 1  
  
# Acceder a elementos de cada array:  
  
print(a[2])  
  
# 3  
  
print(b[1,0])  
  
# 4  
  
print(c[1,0,1])  
  
# 6  
  
# Acceder al último elemento de cada array:  
  
print(a[-1])  
  
# 5  
  
print(b[-1,-1])  
  
# 6  
  
print(c[-1,-1,-1])  
  
# 8
```

Numpy

[:], [::], ...

Acceder a varios elementos de un array.

Para acceder a varios elementos se utiliza el mismo procedimiento que se sigue con [listas](#) y [tuplas](#) en Python: si los elementos son consecutivos se expresa un rango con el índice del elemento inicial seguido de dos puntos ':' y el índice del elemento final:

- `array[0:3]` : Acceder a los 3 primeros elementos (desde el índice 0 al 2).
- `array[1:-2]` : Acceder a los elementos desde el índice 1 al antepenúltimo.

También se puede acceder a elementos separados entre sí por un número fijo de elementos (paso). Para estos casos a la expresión anterior hay que añadir dos puntos ':' seguido del valor que indica el paso o separación entre elementos:

- `array[0:6:2]` : Acceder a los elementos con el índice 0, 2 y 4.
- `array[-1:-4:-2]` : Acceder a los elementos que ocupan la última y antepenúltima posición.

Si se omite en la expresión el índice del elemento inicial la selección comenzará en el primer elemento. En contraposición, si se omite el ordinal del elemento final el acceso terminará en el último elemento:

- `array[:4]` : Acceder a los cuatro primeros elementos (desde el índice 0 al 3).
- `array[2:]` : Acceder a los elementos desde el índice 2 al último.
- `array[:]` : Acceder a todos los elementos.

En referencias de arrays multidimensionales se utilizará el mismo sistema para el acceso a los elementos en cada uno de sus ejes:

- `array[0:2, 0:2]` : Accede a los elementos 0,0; 0,1; 1,0 y 1,1 de un array 2D.
- `array[:, 1:3]` : Accede a todos los elementos del eje 0 pero con índice 1 y 2 en el eje 1.

Cuando se asigna un rango de un array a una variable en realidad se está asignando un array con los elementos seleccionados, en definitiva, un nuevo objeto **ndarray**.

```
# Acceder a varios elementos de un array.

# Declarar 3 arrays con distinto número de dimensiones:
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

# Acceder a los cuatro primeros elementos (del índice 0 al 3):
print(a[0:4])

# [1 2 3 4]

# Acceder a los elementos del 3º hasta el penúltimo (del
```

Numpy

```
# índice 2 al 4):
print(a[2:-1])

# [3 4]

# Acceder a los elementos impares del array:
print(a[:,2])

# [1 3 5]

# Acceder a los elementos pares del array:
print(a[1::2])

# [2 4]

# Acceder a los elementos con índices 0, 3 y 4:
print(a[[0, 3, 4]])

# [1, 4, 5]

# Acceder a los últimos elementos en el eje 0 (filas) y
# a todos en el eje 1 (columnas):
print(b[-1, :])

# [4 5 6]

# Acceder a los últimos elementos en eje 0 y a los del
# índice 1 hasta el final en eje 1:
print(b[-1, 1:])

# [5 6]

# Acceder a todos los elementos en eje 0, en eje 1 y a los
# del índice 1 en adelante en eje 2:
print(c[:, :, 1:])

# [[2]
#    [4]]
#
#    [[6]
#     [8]]]
```

Asignar elementos a una variable.

Numpy

var = []

Asignar un elemento a una variable Numpy.

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
var1 = a[3]
var2 = b[1,2]
var3 = c[0,1,0]
print(var1)

# 4

print(var2)

# 6

print(var3)

# 3

# Obtener el tipo de una variable Numpy:

a = np.array([1, 2, 3, 4, 5])
var1 = a[3]
type(var1)

# numpy.int64
```

var = [:]

Numpy

Asignar una selección de elementos.

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([[1, 2, 3], [4, 5, 6]])
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
var1 = a[3:]
var2 = b[:, -1]
var3 = c[:, 1, :, :]
print(var1)

# [4 5]

print(var2)

# [3 6]

print(var3)

# [[1 2]
#   [3 4]]
```

item()

Asignar un elemento a una variable Python.

```
# Asignar el valor del elemento 1 a una variable Python:

var1 = a.item(1)
print(var1)

# 2

# Asignar el valor del elemento (1,0) a una variable Python:

var2 = b.item(1,0)
print(var2)

# 4

# Obtener el tipo de una variable Python:

type(var1)

# int
```

[condición]

Asignar los elementos que cumplan una condición.

Numpy

```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8]], dtype=np.int8)
b = a[a > 4]
print(b)

# [5 6 7 8]
```

Añadir, insertar y suprimir elementos.

append()

Añadir elementos al final de un array.

```
# Añadir 2 elementos al final de un array:

a = np.array([1, 2, 3, 4, 5])
a = np.append(a, [6, 7])
print(a)

# [1 2 3 4 5 6 7]
```

insert()

Insertar elementos en un array.

```
# Insertar en la posición 3 el valor 0:

a = np.array([1, 2, 3, 4, 5, 6, 7])
a = np.insert(a, 3, 0)
print(a)

# [1 2 3 0 4 5 6 7]

# Insertar en la posición 1 los valores -1 y -2:

a = np.insert(a, 1, [-1, -2])
print(a)

# [ 1 -1 -2  2  3  0  4  5  6  7]

# Insertar columna (eje 1) con valores 0 en la columna
# con índice 2:

b = np.array([[1, 2, 3], [4, 5, 6]])
b = np.insert(b, 2, 0, axis=1)
print(b)

# [[1 2 0 3]
#   [4 5 0 6]]

# Insertar dos columnas (eje 1) con valores -1 y -2
```

Numpy

```
# repetidos al comienzo del array:

b = np.insert(b, 0, [[-1], [-2]], axis=1)
print(b)

# [[-1 -2  1  2  0  3]
#  [-1 -2  4  5  0  6]]

# Insertar columna (eje 1) con valores -4 y -5 al
# comienzo del array:

b = np.insert(b, [0], [[-4], [-5]], axis=1)
print(b)

# [[-4 -1 -2  1  2  0  3]
#  [-5 -1 -2  4  5  0  6]]

# Insertar dos filas (eje 0) con valores 0 en las
# posiciones 0 y 2:

b = np.insert(b, [0, 2], 0, axis=0)
print(b)

# [[ 0  0  0  0  0  0  0]
#  [-4 -1 -2  1  2  0  3]
#  [-5 -1 -2  4  5  0  6]
#  [ 0  0  0  0  0  0  0]]
```

delete()

Suprimir elementos de un array.

```
# Suprimir el elemento 2 del array:

a = np.array([1, 2, 3, 4, 5])
a = np.delete(a, 2)
print(a)

# [1 2 4 5]

# Suprimir la primera fila (eje 0) del array:

b = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
b = np.delete(b, 0, axis=0)
print(b)

# [[ 5  6  7  8]
#  [ 9 10 11 12]]

Suprimir las columnas (eje 1) 0 y 2 del array:

b = np.delete(b, [0, 2], axis=1)
print(b)
```

Numpy

```
# [[ 6  8]
#  [10 12]]
```

delete() y s_[]

Suprimir un rango de elementos.

```
# Suprimir las 2 primeras columnas (eje 1) del array:

b = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
b = np.delete(b, np.s_[0:2], axis=1)
print(b)

# [[ 3  4]
#  [ 7  8]
#  [11 12]]

# Suprimir las columnas (eje 1) pares del array:

b = np.delete(b, np.s_[::2], axis=1)
print(b)

# [[ 4]
#  [ 8]
#  [12]]
```

vstack()

Añadir filas a un array.

```
# Añadir una fila (eje 0).

b = np.array([[1, 2, 3], [4, 5, 6]])
b = np.vstack([b, [7, 8, 9]])
print(b)

# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]
```

hstack()

Añadir columnas a un array.

```
# Añadir una columna (eje 1):
```

Numpy

```
b = np.hstack([b, ([-1], [-2], [-3])])
print(b)

# [[ 1  2  3 -1]
#   [ 4  5  6 -2]
#   [ 7  8  9 -3]]
```

Modificar elementos.

array[indice/s] = valor | itemset()

Modificar un elemento.

```
# Cambiar el valor del elemento 4 por 0:

a = np.array([1, 2, 3, 4, 5])
a[4] = 0 # equivalente a a.itemset(4, 0)
print(a)

# [1 2 3 4 0]

# Cambiar el valor del elemento (1,1) por -1:

b = np.array([[1, 2, 3], [4, 5, 6]])
b[1, 1] = -1 # equivalente a b.itemset((1, 1), -1)
print(b)

# [[ 1  2  3]
#   [ 4 -1  6]]
```

Operaciones con múltiples elementos.

take()

Crear un array a partir de referencias a los elementos de otro array.

Numpy

```
a = np.array([1, 2, 3, 4, 5])
indices = [0, 2, 4]
b = a.take(indices)
print(b)

# [1 3 5]
```

put()

Sustituir valores de un array por nuevos a partir de sus referencias.

```
a = np.array([1, 2, 3, 4, 5])
indices = [1, 3, 4]
nuevos = [10, 30, 40]
a.put(indices, nuevos)
print(a)

# [ 1 10  3 30 40]
```

putmask()

Modificar los elementos del array que cumplan una condición.

```
a = np.array([(1, 2, 3), (4, 5, 6)])
np.putmask(a, a>3, a*2)
print(a)

[[ 1  2  3]
 [ 8 10 12]]
```

repeat()

Crear un array con elementos repetidos.

```
# Crear un array con el mismo elemento repetido:

a = np.repeat(3, 4)
```

Numpy

```
print(a)

# [3 3 3 3]

# Crear un array con varios elementos repetidos:

a = np.repeat([1,2,3], 2)
print(a)

# [1 1 2 2 3 3]

# Crear un array bidimensional repitiendo filas 2 veces:

a = np.repeat([[1,2],[3,4]], 2, axis=0)
print(a)

# [[1 2]
#  [1 2]
#  [3 4]
#  [3 4]]

# Crear un array bidimensional repitiendo columnas 2 veces:

a = np.repeat([[1,2],[3,4]], 2, axis=1)
print(a)

# [[1 1 2 2]
#  [3 3 4 4]]
```

compress()

Seleccionar por su posición elementos de un array.

```
# Seleccionar elementos de "a" con valor 1 en la misma posición:

a = np.array([99, 98, 97, 88, 87, 86, 77, 76, 75])
```

Numpy

```
b = a.compress([0, 0, 1, 0, 0, 1])
print(b)

# [97 86]

# Obtener segmentos de "a" con valor 1 en la misma posición:

a = np.array([[99, 98, 97], [88, 87, 86], [77, 76, 75]])
b = a.compress([1,0,1], axis=0)
print(b)

# [[99 98 97]
#  [77 76 75]]
```

diagonal()

Obtener la diagonal de un array.

```
a = np.arange(9).reshape(3,3)
print(a)

# [[0 1 2]
#  [3 4 5]
#  [6 7 8]]

print(a.diagonal())      # [0 4 8]
print(a.diagonal(-1))    # [3 7]
print(a.diagonal(1))     # [1 5]
```

choose()

Crear un array seleccionando elementos de varios arrays.

```
# Crear un array seleccionando los elementos del segundo array:

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])
```

Numpy

```
matrices = [a, b]
c = np.choose([1], matrices)
print(c)

# [5 6 7 8]

# Crear un array seleccionando elementos de varios arrays.

# Seleccionar el primer elemento del array a; el segundo elemento
# del array b y el tercer elemento del array c.

# matrices es un lista que contiene los arrays a, b, c que ocupan
# las posiciones 0, 1 y 2, respectivamente.

# En el método choose() el primer argumento es una lista que indica
# los elementos que hay que seleccionar. La posición de sus valores
# en dicha lista señalan la posición del elemento a seleccionar y
# el valor el array de donde hay que seleccionar:

a = np.array([10, 20, 30])
b = np.array([40, 50, 60])
c = np.array([70, 80, 90])
matrices = [a, b, c]
d = np.choose([0, 1, 1], matrices)
print(d)

# [10 50 60]

# Seleccionar el primer elemento del array c; el segundo del
# array b y el tercero del array a:

a = np.array([10, 20, 30])
b = np.array([40, 50, 60])
c = np.array([70, 80, 90])
matrices = [a, b, c]
d = np.choose([2, 1, 0], matrices)
print(d)

# [70 50 30]
```

for/in, enumerate(), range()

Recorrer todos los elementos de un array.

```
# Recorrer todos los elementos del vector a con enumerate().

a = np.array([1, 2, 3, 4, 5])
for indice, elemento in enumerate(a):
    print(f'{indice}: {elemento}')
```

Numpy

```
# 0: 1
# 1: 2
# 2: 3
# 3: 4
# 4: 5

# Recorrer todos los elementos de un array 2D con range():

b = np.array([[1, 2, 3], [4, 5, 6]])
filas, columnas = b.shape
for fila in range(filas):
    for columna in range(columnas):
        print(f'{fila},{columna}: {b[fila, columna]}')

# 0,0: 1
# 0,1: 2
# 0,2: 3
# 1,0: 4
# 1,1: 5
# 1,2: 6
```

[for in] | [for in if]

Listas de comprensión.

```
# Generar lista con elementos elevados al cuadrado:

a = np.array([1, 2, 3, 4, 5])
lista1 = [valor**2 for valor in a]
print(lista1)

# [1, 4, 9, 16, 25]

# Generar lista con elementos pares elevados al cuadrado:

lista2 = [valor**2 for valor in a if valor % 2 == 0]
print(lista2)

# [4, 16]
```